

Bàn về lựa chọn thuật toán cho bài toán

Zhang Yunliang

2003

Nguồn gốc: Bàn về lựa chọn thuật toán cho bài toán

IOI China candidate team papers / 2003 / Zhang Yunliang / Zhang Yunliang.doc

Từ khóa

Thuật toán không tối ưu; độ phức tạp lập trình.

Tóm tắt

Bài viết này giới thiệu các thuật toán tốt nhì hoặc thậm chí tương đối kém cho một số bài tin học. Thông qua nhiều ví dụ, bài viết cho thấy khi làm bài không nên chỉ một mực theo đuổi một thuật toán hoàn toàn lý tưởng; trong cài đặt thực tế, các thuật toán tương đối kém có thể có ưu thế rất lớn so với những thuật toán tốt hơn về mặt lý thuyết. Cuối cùng, bài viết rút ra rằng trong luyện tập hằng ngày, ta cần suy nghĩ về một bài toán từ nhiều phía, không nên giữ tâm lý rằng chỉ cần biết thuật toán là xong. Với một bài toán, nên cân nhắc thêm các thuật toán mới; như vậy khi gặp những dạng đề chưa từng thấy, ta sẽ có nhiều hướng suy nghĩ hơn.

1 Dẫn nhập

Thi tin học là một phép kiểm tra tổng hợp về kiến thức máy tính và năng lực lập trình của thí sinh. Trong huấn luyện bình thường, chúng ta thường cố gắng hoàn thiện đến mức cao nhất và tìm cách nghĩ ra thuật toán tốt nhất. Nhưng khi thi, thời gian và trạng thái tâm lý đều khác. Việc viết được chương trình lấy điểm càng nhiều càng tốt trong phòng thi là rất quan trọng. Vì vậy trong luyện tập hằng ngày cần có quan niệm một bài toán có nhiều cách giải: thử qua các thuật toán khác nhau, rồi từ những cách làm tưởng như đơn giản có thể suy ra thuật toán tốt hơn.

Khi học hợp ngữ, thầy giáo từng nói rằng chương trình dịch ra từ hợp ngữ chạy nhanh nhất. Vì sao? Với một lớp bài toán, hoặc để đạt một mục tiêu, thuật toán do con người lập trình càng cần nhiều câu lệnh thì tốc độ chạy của máy càng nhanh; ngược lại, nếu con người bỏ ít công sức lập trình hơn thì tốc độ chạy của máy cũng sẽ chậm hơn. Nếu tìm được một thuật toán có độ khó lập trình thấp mà chương trình vẫn chạy xong trong giới hạn thời gian, thì vừa giảm lao động của con người, vừa viết được chương trình đúng, đồng thời tỉ lệ lỗi khi viết cũng thấp hơn. Do đó trong thi, các thuật toán tốt nhì hoặc kém hơn cũng đáng được cân nhắc.

2 Độ phức tạp của thuật toán

Độ phức tạp mà tôi nói ở đây bao gồm độ phức tạp lập trình, độ phức tạp thời gian và độ phức tạp bộ nhớ. Trong vài giờ thi, độ phức tạp lập trình không thể bị xem nhẹ. Biết thuật toán nhưng không hoàn thành được chương trình là tình huống không lý tưởng nhất. Yêu cầu chất lượng cơ bản nhất đối với mã nguồn là tính đúng đắn và độ tin cậy; miễn là bảo đảm được hai điểm này, nhiều loại thuật toán đều có thể là thuật toán tốt.

Tuy nhiên, độ phức tạp lập trình phụ thuộc vào mức dễ hiểu, dễ kiểm thử và dễ sửa đổi của chương trình. Không thể chương trình nào cũng đạt trạng thái lý tưởng ngay ở lần viết đầu tiên; con người luôn có chỗ sai sót. Nếu có lỗi mà khó kiểm thử đến mức khó phát hiện, hoặc biết lỗi nhưng khó sửa, thì sẽ lãng phí thời gian không cần thiết. Có thể bản thân ta cũng chưa thật quen với thuật toán đã biết, vì ta không thể hiểu thấu đáo mọi phương pháp. Trong tình huống đó, viết chương trình sẽ bất lợi hơn, vì phải tốn nhiều thời gian để hiểu phương pháp ấy. Vì vậy, khi đồng thời xét cả ba loại độ phức tạp này, tìm ra thuật toán thích hợp nhất là điều cần thiết.

3 Một số hướng suy nghĩ cho bài toán

Khi làm bài, thuật toán đầu tiên ta nghĩ ra chưa chắc đã là thuật toán tốt nhất, nhưng điều đó không có nghĩa nó là thuật toán không tốt. Vì vậy, khi nghĩ ra một thuật toán, ta nên xét tính khả thi của nó. Nếu khả thi, chỉ bằng tự cài đặt thử thuật toán đó trước; việc này rèn luyện năng lực viết những thuật toán không chuẩn, và sẽ có ích khi đối diện với các dạng đề mới lạ.

3.1 Ví dụ 1

Mô tả bài toán. Dữ liệu vào gồm:

- một số nguyên dương n , và dãy số nguyên $A_1, A_2, A_3, \dots, A_n$;
- một số nguyên dương m , và dãy số nguyên $B_1, B_2, B_3, \dots, B_m$.

Trong đó

$$1 \leq n \leq 10^6, \quad 1 \leq A_i \leq 10^6, \quad 1 \leq m \leq 1000, \quad 1 \leq B_i \leq 10^6.$$

Dữ liệu ra gồm tổng cộng m dòng, mỗi dòng một số nguyên. Số in ra ở dòng thứ i biểu thị số phần tử trong dãy $\{A\}$ không lớn hơn B_i .

Phân tích thuật toán. Bài này là một dạng suy biến của bài *Mobile Phones* trong IOI. Thuật toán chuẩn dùng cây đoạn để giải; kiến thức cơ bản về cây đoạn có thể xem trong các sách liên quan.

Cách làm tôi muốn nói như sau. Trước hết, ta chú ý đến chênh lệch giữa phạm vi của n và m . Vì vậy ta dùng phương pháp sau. Các mảng ghi nhận là:

- $L[1..1000000]$: ghi nhận, đến thời điểm hiện tại, có $L[k]$ phần tử A_i nhận giá trị k .
- $V[1..1000]$: ghi nhận, đến thời điểm hiện tại, có $V[k]$ phần tử A_i nằm trong $[(k-1)1000+1, k1000]$.

Quy trình chương trình như sau. Đầu tiên đặt các mảng L, V về 0.

1. Đọc n , rồi lần lượt đọc A_i . Với mỗi A_i , giả sử

$$(k-1)1000+1 \leq A_i \leq k1000$$

với k là số nguyên, thực hiện

$$L[A_i] ++, \quad V[k] ++.$$

2. Đọc m , rồi lần lượt đọc B_i . Với mỗi B_i , giả sử

$$(k-1)1000 + 1 \leq B_i \leq k1000$$

với k là số nguyên.

Gọi S là số phần tử trong $\{A\}$ không lớn hơn B_i . Dễ thấy

$$S = \sum_{j=1}^{k-1} V[j] + \sum_{j=(k-1)1000+1}^{B_i} L[j].$$

Sau đó in S .

Phân tích độ phức tạp. Dễ thấy việc tính mỗi giá trị S nhiều nhất cần

$$1000 + 1000 = 2000$$

phép toán. Mỗi lần ghi nhận một A_i cần 2 phép toán. Nhờ đặc điểm phân bố dữ liệu, số phép toán thực tế nhỏ hơn $2000m$.

Đây là phương pháp chia để trị, là một thuật toán ở giai đoạn quá độ, chưa hoàn toàn đầy đủ. Phương pháp này chia thành $1000 \cdot 1000$; từ đó rất dễ liên tưởng đến cách chia thành $100 \cdot 100 \cdot 100$, rồi dần dần suy ra cách làm bằng cây đoạn.

Hãy so sánh độ phức tạp của hai thuật toán:

Cây đoạn	Thuật toán này	Trường hợp xấu nhất
$(n+m) \log_2 G$	$2n + 2000m$	$G = 10^6$

Do đó, chỉ dựa vào trực giác thì ta không thể phán đoán thuật toán nào ưu việt hơn về hiệu suất thời gian. Tuy trong phần lớn tình huống ta không biết chắc kích thước của n và m , nếu có thể xác định cả hai thuật toán đều khả thi thì dùng thuật toán này sẽ giảm độ phức tạp lập trình. Dù sao chương trình cũng do con người viết; về sau nếu cần sửa, việc hiểu cũng thuận tiện hơn.

3.2 Thống kê doanh thu, vòng tuyển chọn Hồ Nam

Mô tả bài toán. Sổ sách của công ty ghi lại doanh thu từng ngày kể từ khi thành lập. Phân tích tình hình kinh doanh là một công việc khá phức tạp: doanh thu sẽ có dao động nhất định, và tất nhiên dao động trong một mức nào đó là chấp nhận được. Nhưng vào một số thời điểm, nếu doanh thu đột ngột tăng rất cao hoặc giảm rất thấp, điều đó chứng tỏ tình hình kinh doanh lúc ấy có vấn đề. Trong quản trị kinh tế, người ta định nghĩa một giá trị dao động tối thiểu để đo lường tình huống này:

$$d_i = \begin{cases} a_1, & i = 1, \\ \min_{1 \leq j < i} |a_i - a_j|, & i > 1. \end{cases}$$

Giá trị dao động tối thiểu càng lớn thì tình hình kinh doanh càng không ổn định. Để phân tích xem tình hình kinh doanh của cả công ty từ khi thành lập đến nay có ổn định hay không, chỉ cần cộng giá trị dao động tối thiểu của từng ngày. Nhiệm vụ của bạn là viết chương trình tính giá trị này. Giá trị dao động tối thiểu của ngày đầu tiên chính là doanh thu của ngày đầu tiên.

Dòng đầu của tệp vào là số nguyên dương n , biểu thị số ngày từ khi công ty thành lập đến hiện tại. n dòng tiếp theo, mỗi dòng có một số nguyên dương a_i , biểu thị doanh thu của công ty trong ngày thứ i . Tệp ra chỉ có một số nguyên dương, tức

$$S = \sum_{i=1}^n d_i.$$

Kết quả nhỏ hơn 2^{31} .

Ví dụ vào-ra:

6

5

1

2

5

4

6

12

Phân tích thuật toán. Ý nghĩa đề bài rất rõ ràng. Điểm then chốt là: khi đọc vào một số, ta phải tìm trong các số đã đọc trước đó số có chênh lệch nhỏ nhất với số này.

Thuật toán chuẩn của bài dùng cây cân bằng. Cây cân bằng là một loại cây tìm kiếm có thể tự thích ứng để làm cho độ sâu của cây nhỏ nhất có thể; chi tiết có thể tham khảo các sách liên quan.

Tiền xử lý. Đọc toàn bộ dữ liệu, sắp xếp $\{a\}$ tăng dần, thu được dãy

$$B_1, B_2, \dots, B_n.$$

Gán bộ cộng dồn S bằng 0.

Quy trình chính. Đọc dữ liệu cần xử lý, xử lý doanh thu của từng ngày và tính giá trị dao động tối thiểu của ngày đó.

Ở đây V, L được định nghĩa giống ví dụ 1, nhưng lần này dùng

$$L[1..32767], \quad V[1..327],$$

tức là thay đổi phạm vi và chia các khoảng theo đơn vị độ dài 100.

Đọc doanh thu trong ngày P . Dùng tìm kiếm nhị phân trong dãy $\{B\}$ để tìm $B_q = P$. Giả sử

$$(k-1)100 + 1 \leq q \leq k100.$$

Nếu $V[k] > 0$, tức trong khoảng này đã có dữ liệu được ghi nhận, thì trong khoảng này tìm số nhỏ nhất g_a thỏa $g_a \geq q$ và $L[g_a] > 0$, đồng thời tìm số lớn nhất g_b thỏa $g_b \leq q$ và $L[g_b] > 0$.

Nếu không tìm được g_a hoặc g_b , xét trường hợp không tìm được g_b như sau; trường hợp g_a tương tự. Lần lượt tra $V[k-1], V[k-2]$ cho đến $V[1]$, tìm một t sao cho $V[t] > 0$. Nếu tìm được t như vậy, thì trong khoảng

$$((t-1)100 + 1, t100)$$

tìm số lớn nhất h sao cho $L[h] > 0$, rồi đặt $g_b = h$.

Sau đó tính giá trị dao động tối thiểu của ngày hiện tại. Nếu tìm được g_a hoặc g_b , so sánh hiệu giữa P và B_{g_a} , hoặc giữa P và B_{g_b} , lấy giá trị nhỏ hơn làm giá trị dao động tối thiểu của ngày hiện tại. Nếu đồng thời không tìm được cả g_a lẫn g_b , điều đó chứng tỏ đây là ngày đầu tiên, và giá trị dao động tối thiểu chính là P .

Cộng giá trị dao động tối thiểu của ngày hiện tại vào bộ cộng dồn S . Đưa phần tử q vào mảng, tức thực hiện

$$L[q] ++, \quad V[k] ++.$$

Kết thúc. In S .

Ở đây V, L được định nghĩa giống ví dụ 1. Lần này dùng

$$L[1..32767], \quad V[1..327],$$

tức là chỉ thay đổi phạm vi và lấy độ dài 100 làm đơn vị chia khoảng.

Chương trình chi tiết xem `myturn.pas` (có cải tiến đôi chút). Chương trình chuẩn xem `Turnover.pas`. Tuy độ dài chương trình đại khái tương đương, việc kiểm thử và sửa đổi thuật toán này lại dễ hơn cây cân bằng rất nhiều, vì nó được chia thành hai bước, có thể kiểm thử từng bước riêng, và yêu cầu kỹ thuật của mỗi bước đều không cao.

4 Tổng kết

Mỗi thuật toán đều có ưu thế riêng. Chẳng hạn với sắp xếp, khi dữ liệu nhỏ ta dùng nổi bọt, chọn trực tiếp và những thuật toán có độ phức tạp thời gian cao nhưng độ phức tạp lập trình thấp; ngược lại, ta sẽ dùng sắp xếp vun đống, sắp xếp nhanh, sắp xếp trộn, v.v. Hiệu quả của mỗi thuật toán là khác nhau trong những hoàn cảnh khác nhau.

Trong quá trình lập trình và trong luyện tập hằng ngày, ta cần suy nghĩ về một bài toán từ nhiều phía, không nên giữ tâm lý rằng biết thuật toán là xong. Với một bài toán, nên cân nhắc thêm các thuật toán mới; như vậy ta có thể phát hiện hoàn cảnh sử dụng của từng thuật toán, và khi gặp những dạng đề chưa từng thấy, ta sẽ có nhiều hướng suy nghĩ hơn.

Tài liệu tham khảo

1. Fu Qingxiang, Wang Xiaodong biên soạn. *Thuật toán và cấu trúc dữ liệu*. Nhà xuất bản Công nghiệp Điện tử.
2. Wu Wenhui, Wang Jiande biên soạn. *Phân tích thuật toán thực dụng và thiết kế chương trình*. Nhà xuất bản Công nghiệp Điện tử.